Cloud Retail Data Lake & Data Warehouse

(Python-Based ETL and Azure Star-Schema Analytics Platform)

Digital Egypt Pioneers Initiative (DEPI)

Round 5 — Data Engineering Track

Project No. 3

Microsoft Azure Cloud Platform

Project Documentation — 2026

Presented by:

Digital Egypt Pioneers Initiative (DEPI) — Round 5, Data Engineering Track — Project No. 3

#	Name	Role
1	Seif Farid Shawky El-Nemr	Team Leader
2	Mohamed Tarek Abdel-Moneim Maharem	Member
3	Kirollos Magdy Milad Hanna	Member
4	Mohamed Fawzy Abu El-Magd El-Asour	Member
5	Adel Mohamed Abdel-Mawgoud Mohamed	Member
6	Nourhan Mahmoud Mohamed Abdel-Latif	Member

Project: **Cloud Retail Data Lake & Data Warehouse**

Cloud platform: Microsoft Azure

Dataset: Maven Fuzzy Factory toy-store e-commerce dataset (six tables, approximately 1.74 million records, March 2012 – March 2015)

Acknowledgments

The project team wishes to express its sincere gratitude to the Digital Egypt Pioneers Initiative (DEPI) for providing the Data Engineering track and the learning environment in which this project was carried out.

The team also thanks the DEPI technical instructors for their guidance, feedback, and review throughout the planning, analysis, and design phases of the project.

Thanks are extended to Microsoft for the Azure Free Trial and Azure for Students programs, whose credits make it possible to build and evaluate a cloud data platform within a student budget, and to Maven Analytics for publishing the Maven Fuzzy Factory toy-store dataset that serves as the project's data source.

Finally, the team thanks its families and colleagues for their patience and support, and each team member for the shared effort invested in this work.

Abstract

This document presents the planning, analysis, and design of the project **Cloud Retail Data Lake & Data Warehouse**, undertaken within Round 5 of the Digital Egypt Pioneers Initiative (DEPI) in the Data Engineering track. The project designs a cloud data platform on Microsoft Azure for an online retail store, in which raw sales and web-traffic files are stored in a data lake, cleaned and validated, modeled into a star schema, and loaded into a warehouse for analysis and reporting.

The data source is the Maven Fuzzy Factory toy-store e-commerce dataset, which contains six tables and approximately 1.74 million records covering March 2012 to March 2015. Because the original data is clean, a reproducible script injects 32 documented types of data-quality issues. This gives the pipeline real cleaning work and gives the tests a known answer key against which every detected and rejected record can be checked.

The design stores data in Azure Data Lake Storage Gen2 using raw, processed, and curated zones. A Python-based ETL pipeline built on Python, pandas, and NumPy cleans, validates, and transforms the data and generates analysis-ready Parquet files. The curated data is modeled as a Kimball star schema at order-line grain with the fact tables FactSales and FactRefunds and the dimensions DimDate, DimProduct, DimCustomer, and DimStore. Azure Synapse Serverless SQL provides a low-cost query layer over the lake, and the final model is loaded into a warehouse that stores data. The pipeline is orchestrated and monitored, and access is secured with Microsoft Entra ID and role-based access control, while total cloud spend is kept within the team's Azure credit.

The report covers the project introduction, background and related work, project planning and system analysis, and system design. The implementation and testing chapter describes the planned implementation approach and test plan; execution results are to be added as the implementation phase is completed and verified.

Table of Contents

LIST OF FIGURES

Fig (3.1): Use-Case Diagram (UC-01 to UC-07) 27
Fig (4.1): System Architecture Diagram 31
Fig (4.2): Data Lake Zones and Data Flow 33
Fig (4.3): Star Schema Diagram 36
Fig (4.4): Pipeline Orchestration Flow 37
Fig (4.5): Entity Relationship Diagram of the Source Dataset 38
Fig (4.6): Data Flow Diagram (Level 0 and Level 1) 38
Fig (4.7): UML Sequence Diagram of an End-to-End Pipeline Run 39

LIST OF TABLES

Table (1.1): Project Information 9
Table (1.2): Project Team Members 10
Table (1.3): Project Scope 12
Table (2.1): Mapping of Medallion Layers to Project Zones 15
Table (2.2): Comparison of Related Solutions with This Project 18
Table (3.1): Project Timeline and Milestones 20
Table (3.2): Gantt Chart Table 21
Table (3.3): Team Roles and Responsibilities 22
Table (3.4): Resource Allocation 22
Table (3.5): Feasibility Assessment 23
Table (3.6): Risk Assessment Matrix 23
Table (3.7): Stakeholders and Their Needs 24
Table (3.8): User Requirements 24
Table (3.9): Functional Requirements 25
Table (3.10): Non-Functional Requirements 25
Table (3.11): System and Domain Requirements 26
Table (3.12): Use Cases 27
Table (3.13): Key Performance Indicators 28
Table (4.1): Architecture Layers and Components 30
Table (4.2): Source Dataset Tables 32
Table (4.3): Data Lake Zones 32
Table (4.4): Python-Based Cleaning and Transformation Stages 34
Table (4.5): Data Quality Rule Categories 35
Table (4.6): Fact Tables (Proposed) 35
Table (4.7): Dimension Tables (Proposed) 36
Table (4.8): Roles of the Curated Zone, Serverless SQL, and the Warehouse 37

Table (4.9): Security Controls 38
Table (5.1): Implementation Status Register 41
Table (5.2): Test Plan 43
Table (5.3): Performance and Cost Results (to be completed) 43

LIST OF ABBREVIATIONS

ADLS Gen2	Azure Data Lake Storage Gen2	**FR**	Functional Requirement
BI	Business Intelligence	**KPI**	Key Performance Indicator
COGS	Cost of Goods Sold	**NFR**	Non-Functional Requirement
CSV	Comma-Separated Values	**PoC**	Proof of Concept
DEPI	Digital Egypt Pioneers Initiative	**RBAC**	Role-Based Access Control
DFD	Data Flow Diagram	**SQL**	Structured Query Language
DW	Data Warehouse	**T-SQL**	Transact-SQL
ELT	Extract, Load, Transform	**UC**	Use Case
ERD	Entity Relationship Diagram	**UML**	Unified Modeling Language
ETL	Extract, Transform, Load	**USD**	United States Dollar

LIST OF CONTENTS

CHAPTER 1 Introduction 9
1.1 Project Overview 9
1.2 Problem Statement 10
1.3 Motivation 10
1.4 Project Objectives 11
1.5 Project Scope 11
1.6 Project Roadmap 12
CHAPTER 2 Background and Previous Work 14
2.1 Introduction 14
2.2 Data Engineering Fundamentals 14
2.3 Data Lakes and Data Warehouses 14
2.4 ETL and ELT Concepts 15
2.5 Python, pandas, and NumPy for Data Engineering 15
2.6 Azure Data Lake Storage Gen2 16
2.7 Parquet File Format 16
2.8 Dimensional Modeling and Star Schema 16
2.9 Azure Synapse Serverless SQL 16
2.10 Related Work and Existing Solutions 17
2.11 Comparative Analysis 17
2.12 Proposed Solution and Research Gap 18
CHAPTER 3 Project Planning and System Analysis 20
3.1 Project Planning and Management 20
3.1.1 Project Timeline and Milestones 20
3.1.2 Gantt Chart 20
3.1.3 Team Roles and Responsibilities 21

3.1.4 Resource Allocation 22
3.1.5 Feasibility Study and Estimated Cost 22
3.2 Risk Assessment and Mitigation 23
3.3 Stakeholder Analysis 24
3.4 Requirements Gathering 24
3.4.1 User Requirements 24
3.4.2 Functional Requirements 25
3.4.3 Non-Functional Requirements 25
3.4.4 System and Domain Requirements 26
3.5 Use Cases and User Stories 26
3.5.1 User Stories 26
3.5.2 Use Cases 27
3.6 Key Performance Indicators 27
CHAPTER 4 System Design 30
4.1 System Architecture Overview 30
4.2 Design Constraints and Assumptions 31
Constraints 31
Assumptions and dependencies 31
4.3 Data Source and Dataset Description 32
4.4 Data Lake Architecture 32
4.4.1 Raw Zone 33
4.4.2 Processed Zone 33
4.4.3 Curated Zone 33
4.5 Data Ingestion Design 33
4.6 Python-Based Data Cleaning and Transformation Design 33
4.7 Data Quality Validation Framework 34
4.8 Dimensional Modeling 35
4.8.1 Fact Tables 35
4.8.2 Dimension Tables 36
4.8.3 Star Schema Design 36
4.9 Warehouse Architecture 36
4.10 Pipeline Orchestration 37
4.11 Security and Role-Based Access Control 37
4.12 System Diagrams 38
CHAPTER 5 Implementation and Testing 41
5.1 Development Environment 41
5.2 Python ETL Implementation 41
5.3 Data Cleaning with pandas 42
5.4 Numerical Processing with NumPy 42
5.5 Data Validation and Rejected Records 42
5.6 Parquet File Generation 42
5.7 Azure Data Lake Integration 42
5.8 SQL Transformation and Warehouse Loading 42
5.9 Pipeline Monitoring 42
5.10 Testing and Validation 43
5.11 Performance and Cost Evaluation 43
REFERENCES 44

CHAPTER ONE

CHAPTER 1 Introduction

1.1 Project Overview

The project builds a cloud data platform on Microsoft Azure for an online retail store. Raw sales and web-traffic files land in a data lake, are cleaned and modeled into a star schema, and are loaded into a warehouse for analysis and reporting. The platform is delivered as part of the Digital Egypt Pioneers Initiative (DEPI), Round 5, in the Data Engineering track, as Project No. 3.

The data source is the Maven Fuzzy Factory toy-store dataset [7]. It contains 6 tables and approximately 1.74 million records covering March 2012 to March 2015. The original data is clean, so the team injects documented data-quality issues with a reproducible script. This gives the pipeline real cleaning work and gives the tests a known answer key.

Data cleaning, validation, and transformation are implemented as a Python-based ETL pipeline that uses Python, pandas, and NumPy, and that generates analysis-ready Parquet files. The general information about the project is summarized in Table (1.1), and the project team is listed in Table (1.2).

Table (1.1): Project Information

Item	Details
Program	Digital Egypt Pioneers Initiative (DEPI), Round 5
Track	Data Engineering
Project	No. 3 — Cloud Retail Data Lake & Data Warehouse
Cloud platform	Microsoft Azure
Dataset	Maven Fuzzy Factory toy-store e-commerce dataset; 6 tables; approximately 1.74 million records (1,735,068); March 2012 – March 2015
Data processing technologies	Python, pandas, and NumPy (Python-based ETL pipeline)
Team size	6 members
GitHub repository	github.com/Muhammad-Maharem/Cloud-Retail-Data-Lake-and-Data-Warehouse
Source documentation	Part 1 — Planning & Management, Literature Review, Requirements Gathering

Table (1.2): Project Team Members

#	Name	Role
1	Seif Farid Shawky El-Nemr	Team Leader
2	Mohamed Tarek Abdel-Moneim Maharem	Member
3	Kirollos Magdy Milad Hanna	Member
4	Mohamed Fawzy Abu El-Magd El-Asour	Member
5	Adel Mohamed Abdel-Mawgoud Mohamed	Member
6	Nourhan Mahmoud Mohamed Abdel-Latif	Member

1.2 Problem Statement

An online retail store continuously produces sales records and web-traffic records. When these records remain as separate raw files, they are difficult to analyze reliably: the files can contain duplicate rows, inconsistent date, price, text, and boolean formats, missing values, values that break business rules (for example, negative quantities or refunds larger than the item price), and records that refer to orders or products that do not exist.

Store management, marketing analysts, and BI developers need a single, trusted, documented model from which they can obtain figures such as monthly revenue, profit, refunds, and conversion rate. Building such a model on a cloud platform raises two further problems. First, it must be possible to prove that the cleaning logic catches every defect, which is hard when the source data has no known defects. Second, the platform must be built and operated within a student budget, so always-on and expensive services must be avoided.

The project therefore addresses the following question: *how can raw retail files be ingested, cleaned, validated, modeled, and served on Azure as a trusted star schema, with measurable data quality and strict cost control?*

1.3 Motivation

Cloud platforms have become the usual place to build analytics solutions, and the data lake and data warehouse patterns are central skills for a data engineer. A retail dataset is a realistic and accessible domain: it combines transactional data (orders, order items, refunds) with behavioral data (website sessions and page views), and the questions asked of it are easy to state and to verify.

The project combines Azure services with open-source Python tools. Python, pandas, and NumPy are widely used for batch data cleaning and transformation, and are well suited to a dataset of about 1.74 million records processed in scheduled batches. SQL is used for the query

layer and for the warehouse, so the team practices the full path from raw files to analytical queries.

Two features distinguish the project from the reference designs reviewed in Chapter Two. The first is **measurable data quality**: because the team injects a known set of issues, the pipeline can be tested against an answer key instead of being judged by inspection. The second is **cost discipline**: the whole platform is designed to run inside the team's Azure credit, using Free Trial credit for learning and the proof of concept and Azure for Students credit for the final implementation.

1.4 Project Objectives

The objectives of the project are:

1. Store raw data in Azure Data Lake Storage Gen2, organized into raw, processed, and curated zones.
2. Clean, validate, and transform the data using Python, pandas, and NumPy, and generate analysis-ready Parquet files for efficient storage, querying, and downstream analytics.
3. Detect and handle every injected data-quality issue, with rejected records logged and counted.
4. Model the data as a Kimball star schema at order-line grain.
5. Expose the data through Synapse Serverless SQL and load the final model into a warehouse.
6. Orchestrate and monitor the pipeline, and secure it with role-based access control.
7. Keep total cloud spend inside the team's Azure credit.

1.5 Project Scope

Table (1.3) lists what is in and out of scope. Items marked as optional are pursued only if time allows.

Table (1.3): Project Scope

In scope	Out of scope
Batch ingestion of CSV files into ADLS Gen2	Real-time or streaming ingestion
Cleaning, validation, and transformation with Python, pandas, and NumPy	Machine-learning models
Star schema: fact and dimension tables	Building a customer-facing application or UI
Synapse Serverless SQL, external tables and views	Production deployment or a service-level agreement
Loading the warehouse and sample analytical queries	Integrating live company systems
Pipeline orchestration, monitoring, and RBAC	Delta Lake and Z-ordering (optional, only if time allows)

1.6 Project Roadmap

This document follows the structure of the graduation project template and is organized into five chapters and a reference list:

- **Chapter One — Introduction** presents the project overview, problem statement, motivation, objectives, scope, and roadmap.
- **Chapter Two — Background and Previous Work** introduces the concepts used in the project (data lakes and warehouses, ETL, Python data processing, ADLS Gen2, Parquet, dimensional modeling, Synapse Serverless SQL), reviews similar solutions, and states the research gap.
- **Chapter Three — Project Planning and System Analysis** covers the timeline, Gantt chart, roles, resources, feasibility and cost, risks, stakeholders, requirements, use cases, and KPIs.
- **Chapter Four — System Design** describes the architecture, data lake zones, ingestion, Python-based cleaning and transformation, the data-quality framework, dimensional model, warehouse, orchestration, and security.
- **Chapter Five — Implementation and Testing** describes the development environment, the planned implementation of each component, and the test plan. Results are reported only when they have been verified.

Document status. The planning, literature review, and requirements sections originate from the team's first submission (Part 1) and have been reorganized without removing technical information. The design chapter describes the planned architecture. Where a diagram or a result has not yet been produced, the document contains a clearly labeled placeholder, and the text distinguishes planned work from verified implementation.

CHAPTER TWO

CHAPTER 2 Background and Previous Work

2.1 Introduction

This chapter introduces the concepts on which the project is built and reviews existing Azure data platforms. Existing platforms already agree on the core pattern: layered storage in a data lake, a processing engine for transformation, and a SQL layer for reporting [1][4]. The project follows that pattern and adds measurable data-quality testing and strict cost control, which the solutions reviewed here leave out.

2.2 Data Engineering Fundamentals

Data engineering is concerned with moving data from its sources to the places where it can be analyzed, and with making it trustworthy on the way. Typical tasks are ingesting data, storing it, cleaning and validating it, modeling it for analysis, serving it to users, and orchestrating, monitoring, and securing these steps. Quality and traceability are central: an analyst must be able to trust a figure and, if needed, trace it back to its source record.

In this project the data engineering work is divided into six roles (Chapter Three) that map to the main stages of the pipeline: platform and storage, Python data processing, SQL and dimensional modeling, quality and testing, and documentation and version control, led by a team leader.

2.3 Data Lakes and Data Warehouses

A **data lake** stores data in its original or lightly processed form, typically as files in cheap, scalable storage, and does not require a schema to be enforced when data is written. A **data warehouse** stores curated, structured data that is modeled for analytical queries and supports transactional loading of tables. The two are complementary: the lake is the landing and processing area, and the warehouse is the trusted serving layer.

A widely used way to organize a lake is the **medallion (multi-zone) architecture**. Data moves through three layers of rising quality: Bronze keeps data exactly as it arrives, Silver fixes errors, standardizes formats, and removes duplicates, and Gold holds curated data for reports [1]. Table (2.1) shows how the project's zones map to these layers.

Table (2.1): Mapping of Medallion Layers to Project Zones

Medallion layer	Project zone	Content in this project
Bronze	Raw	The six source CSV files, stored unchanged
Silver	Processed	Cleaned and validated tables written as Parquet, plus rejected-record logs
Gold	Curated	Fact and dimension tables of the star schema, written as Parquet

2.4 ETL and ELT Concepts

ETL (extract, transform, load) transforms data before it is loaded into its target, whereas **ELT** (extract, load, transform) loads data first and transforms it inside the target platform. The project uses both ideas in different places. Raw files are first loaded unchanged into the lake (the "load"-first idea of ELT). The cleaning and validation step is then performed by a Python-based ETL pipeline that reads the raw files, applies transformations, and writes processed and curated Parquet files. Finally, SQL scripts expose the curated data through external tables and views and load the warehouse.

2.5 Python, pandas, and NumPy for Data Engineering

Python is a general-purpose language with a large ecosystem for data work. **pandas** provides the DataFrame, a table-like in-memory structure with operations for reading CSV files, filtering, joining, grouping, de-duplicating, type conversion, and handling missing values. **NumPy** provides fast array operations and numeric functions that pandas builds upon, and can be used directly for vectorized conditions and numeric checks.

These tools are appropriate for the project's batch-processing requirements for the following reasons:

- The workload is **batch only**; real-time and streaming ingestion are out of scope (Table 1.3).
- The dataset has approximately 1.74 million records spread over six tables, so each table can be read, cleaned, and written as a separate step. The expected per-table size is to be confirmed during dataset profiling in Phase 2.
- Cleaning rules such as de-duplication, format standardization, null handling, and business-rule checks map directly to DataFrame and NumPy operations, which keeps the rules readable and testable.
- A Python-based ETL pipeline does not require provisioning a distributed processing cluster, which helps keep the platform inside the student budget.

Two limitations are acknowledged. First, pandas processes data in memory on a single machine, so the design processes one table at a time and the approach would need to be revisited if the data volume grew substantially. Second, pandas and NumPy do not by themselves write Parquet files: writing Parquet requires a compatible engine, such as PyArrow, which is installed alongside them. The exact library versions and the memory and run-time behavior of the pipeline are subject to verification during implementation.

2.6 Azure Data Lake Storage Gen2

Azure Data Lake Storage Gen2 (ADLS Gen2) is the storage service used for the data lake. It provides scalable file storage organized in containers and folders that can be queried by Azure services, including Synapse Serverless SQL. In the project, a single ADLS Gen2 account holds the raw, processed, and curated zones, and access to it is controlled through Microsoft Entra ID and role-based access control.

2.7 Parquet File Format

Apache Parquet is a columnar file format that stores data by column, supports compression, and keeps the schema together with the data. Columnar storage reduces the amount of data read by queries that need only some columns. This matters in the project because Synapse Serverless SQL charges only for the data each query processes [3]. The processed and curated zones are therefore written as Parquet, while the raw zone keeps the original CSV files.

2.8 Dimensional Modeling and Star Schema

Kimball dimensional modeling organizes analytical data into **fact tables**, which hold measurements of business events, and **dimension tables**, which hold the descriptive context (who, what, when, where). Kimball describes declaring the grain as "the pivotal step in a dimensional design" and recommends the atomic grain, because atomic data "withstands the assault of unpredictable user queries" [2]. The project therefore uses one row per product per order (the order line) as the grain of the sales fact table. A star schema places each fact table at the center and joins it to its dimensions through keys.

2.9 Azure Synapse Serverless SQL

Synapse Serverless SQL queries Parquet and CSV files in the lake with T-SQL, without loading them, and charges only for the data each query processes. It **cannot store tables or run INSERT, UPDATE, or DELETE statements** [3]. It is therefore a cheap layer for exploration and for external tables and views over the lake, but it is a query layer over external data and not

a conventional database. The final model still needs a warehouse that stores data, and the data lake itself must not be confused with that physical warehouse.

2.10 Related Work and Existing Solutions

Three existing solutions were reviewed. Their descriptions of Spark-based processing are retained here because they are accurate descriptions of those external architectures; they do not describe the processing technology of this project.

- **Analytics end-to-end with Microsoft Fabric** [4] is a Microsoft reference architecture with ingest, store, process, and serve stages on OneLake, medallion layers, pipelines, Spark, a warehouse, and Power BI. It is complete and governed and covers streaming, machine learning, and CI/CD, but it is built on Fabric capacity billed hourly and has many overlapping services.
- **Azure Synapse Analytics end-to-end deployment accelerator** [5] provides Bicep templates that deploy Synapse, Spark and SQL pools, data lake tiers, and Purview, with RBAC wired automatically. It offers infrastructure as code, but deploys many paid services at once and includes no dataset, model, or quality rules.
- **End-to-end data engineering project on Azure** [6] is a community project (2025) that uses Data Factory ingestion, ADLS Gen2, Databricks with PySpark, bronze/silver/gold layers, Synapse serverless views, and Power BI. It is a realistic, low-cost stack close to this project's, and uses Key Vault for credentials, but it documents no data-quality checks and no testing.

2.11 Comparative Analysis

Table (2.2) compares the reviewed solutions with this project. The rows describing external solutions keep their original processing technology (Spark, Databricks) because that is how those solutions are built. The last row describes this project's own approach.

Table (2.2): Comparison of Related Solutions with This Project

Solution	What it does	Strengths	Limitations for our case
Analytics end-to-end with Microsoft Fabric (Microsoft reference architecture) [4]	Ingest, store, process, and serve stages on OneLake, with medallion layers, pipelines, Spark, a warehouse, and Power BI	Complete and governed design; covers streaming, ML, and CI/CD	Built on Fabric capacity billed hourly; many overlapping services; far beyond a $100 budget
Azure Synapse Analytics end-to-end deployment accelerator [5]	Bicep templates that deploy Synapse, Spark and SQL pools, data lake tiers, and Purview, with RBAC wired automatically	Infrastructure as code; permissions configured for you	Deploys many paid services at once; no dataset, model, or quality rules included
End-to-end data engineering project on Azure (community project, 2025) [6]	Data Factory ingestion, ADLS Gen2, Databricks PySpark, bronze/silver/gold, Synapse serverless views, Power BI	Realistic, low-cost stack close to ours; uses Key Vault for credentials	No documented data-quality checks and no testing
This project	ADLS Gen2 zones, Python-based cleaning with pandas and NumPy, Parquet output, Kimball star schema, Serverless SQL, warehouse load, pipeline monitoring	Measured quality: every injected issue is known and tested; built to a student budget	Batch only; single online store; small product catalog (4 products)

2.12 Proposed Solution and Research Gap

The reviewed solutions share a layered lake, a processing step, and a SQL serving layer, but none of them combines a known answer key for data quality with a budget a student team can carry. The proposed solution takes what is useful from them and differs where it matters:

- **We take** the three-zone lake from the medallion pattern [1], atomic-grain modeling from Kimball [2], and Serverless SQL as the low-cost query layer [3].
- **We differ** by testing data quality against a known answer key. The Maven Analytics toy-store dataset [7] (1,735,068 records) is clean, so a seeded script injects 32 documented issue types, and the pipeline must catch all of them.
- **We differ** in the processing technology: transformation is done by a Python-based ETL pipeline using pandas and NumPy, rather than by a distributed Spark engine, which is sufficient for batch processing of a dataset of this size and avoids the cost of a processing cluster. This is a design rationale to be confirmed by the measurements in Chapter Five.
- **We avoid** costly always-on services such as a Dedicated SQL Pool, which the reference designs assume, and keep the whole platform in one resource group with budget alerts.

CHAPTER THREE

CHAPTER 3 Project Planning and System Analysis

3.1 Project Planning and Management

The platform must turn messy raw files into a trusted star schema that analysts can query, at a cost a student budget can carry. This section presents the plan used to achieve that.

3.1.1 Project Timeline and Milestones

The project is divided into four phases with official submission dates, shown in Table (3.1).

Table (3.1): Project Timeline and Milestones

Phase	Dates	Main deliverables
1. Planning, Literature Review, Requirements	Until 19 Oct 2026	This document; GitHub repository
2. System Analysis & Design	20 Oct – 6 Nov 2026	Dataset profiling, star schema, architecture diagram, test plan, Azure proof of concept
3. Implementation	7 – 30 Nov 2026	Data lake, Python ETL scripts (pandas and NumPy), SQL scripts, warehouse load, pipeline, access control
4. Testing & Final Presentation	1 – 4 Dec 2026	Test results, user and technical documentation, presentation

The critical path is dataset approval, then the grain, then the star schema, then the proof of concept. A delay in any of these delays everything after it.

3.1.2 Gantt Chart

Table (3.2) shows the schedule by week (weeks start on Monday). Shaded cells mark active weeks, and ◆ marks an official submission. The owner column refers to the role numbers in Table (3.3).

Table (3.2): Gantt Chart Table

Task	Owner	Oct 5	Oct 12	Oct 19	Oct 26	Nov 2	Nov 9	Nov 16	Nov 23	Nov 30
Planning, literature review, requirements	1, 2, 3, 4, 5	■	■	◆						
GitHub repository and structure	6	■	■							
Dataset approval and profiling	3			■						
Grain and star schema design	4			■	■					
Architecture, ERD, DFD and UML diagrams	2, 4, 6			■	■					
Test plan and data-quality rules	5			■	■					
Azure proof of concept	2, 3				■	■				
System Analysis & Design submission (6 Nov)	1, 6					◆				
Data lake zones, raw upload, Serverless SQL queries	2, 4					■	■			
Python (pandas and NumPy) cleaning and data-quality checks	3, 5						■	■		
Star schema build and warehouse load	3, 4							■	■	
Pipeline, monitoring and access control	2								■	
Analytical queries	4								■	
Implementation submission (30 Nov)	All									◆
Testing, reports and presentation (4 Dec)	All									◆

3.1.3 Team Roles and Responsibilities

Each of the six members holds one role. The Team Leader is assigned; the remaining roles are to be assigned to the other five members. Each member works on a personal branch and merges through pull requests, so every member has commits under their own name.

Table (3.3): Team Roles and Responsibilities

#	Role	Main responsibilities	Name
1	Team Lead	Meetings, deadlines, contact with the instructor, submissions	Seif Farid Shawky El-Nemr
2	Azure & Data Lake Engineer	Azure subscription and cost, ADLS Gen2, Synapse workspace, access control, pipeline	To be assigned
3	Python Data Processing Engineer	Dataset profiling, Python (pandas and NumPy) cleaning and transformation scripts, Parquet output	To be assigned
4	SQL & Data Modeling Engineer	Grain, star schema, external tables and views, warehouse load, analytical queries	To be assigned
5	Data Quality & Testing Engineer	Quality rules, rejected records, test plan, test cases, bug reports	To be assigned
6	Docs & GitHub Engineer	Repository structure, branching, README, PDF compilation, screenshots, presentation	To be assigned

3.1.4 Resource Allocation

Table (3.4): Resource Allocation

Resource	Allocation	Used for
Team members	6 members, one role each (Table 3.3)	All phases
Azure Free Trial	$200 credit, until about 9 Nov 2026	Learning and the proof of concept
Azure for Students	$100 credit	Final implementation and the presentation
Azure services	ADLS Gen2, Synapse Serverless SQL, Synapse or Data Factory pipelines, Azure SQL Database, Microsoft Entra ID	Storage, query layer, warehouse, orchestration, access control
Data	Maven Fuzzy Factory dataset (6 tables, 1.74M records) and the dirty-data generator script	Source data and test answer key
Tools	GitHub, Python (pandas, NumPy, and a Parquet-compatible library such as PyArrow), SQL, Power BI	Code, version control, data processing, analysis, and reporting

3.1.5 Feasibility Study and Estimated Cost

The feasibility of the project is assessed along four dimensions in Table (3.5). The assessment is qualitative and is based on the plan, the available credits, and the risks identified in Section 3.2.

Table (3.5): Feasibility Assessment

Dimension	Assessment
Technical	The required capabilities (ADLS Gen2, Synapse Serverless SQL, pipelines, Azure SQL Database, and Python data processing) are available on Azure and in open-source libraries. Availability under the student subscription quotas is checked early by the Azure proof of concept; Azure SQL Database is the fallback warehouse.
Economic	The cost target is to stay within the $100 Azure for Students credit. The $200 Free Trial credit, which expires about 9 Nov 2026, is used only for learning and the proof of concept. Budget alerts at 50%, 80%, and 100%, no Dedicated SQL Pool, and auto-pausing of idle compute control the spend.
Schedule	Four milestones with fixed dates between 19 Oct and 4 Dec 2026. The critical path (dataset approval, grain, star schema, proof of concept) is front-loaded in Phase 2.
Operational	Six team members with defined roles, one branch per member, pull-request reviews, and each task paired with a backup member reduce the effect of a member becoming unavailable.

Estimated cost. The available budget is $300 in total credit ($200 Free Trial for learning and the proof of concept, $100 Azure for Students for the final implementation), and the design target is that the final build and presentation stay within the $100 student credit. A per-service cost estimate is not yet available; it is to be prepared from the proof-of-concept measurements and checked with the Azure Cost Management report.

3.2 Risk Assessment and Mitigation

Table (3.6) lists the main project risks with their impact, likelihood, and mitigation.

Table (3.6): Risk Assessment Matrix

Risk	Impact	Likelihood	Mitigation
Azure credit runs out before the final presentation	High	Medium	Budget alerts at 50%, 80%, and 100%; no Dedicated SQL Pool; auto-pause on SQL and other pausable compute; one resource group so everything can be deleted at once
Free Trial expires (about 9 Nov 2026) in the middle of the project	Medium	High	Trial used only for learning and the proof of concept; final build on the Azure for Students subscription
Student subscription quotas block Synapse or another required service	High	Medium	Proof of concept checks every service early; Azure SQL Database as a fallback warehouse
Cleaning rules miss injected issues	Medium	Medium	Injection manifest used as the answer key; tests require 100% of injected issues to be caught
Dataset has no physical stores	Medium	High	Store dimension modeled as the online sales channel (traffic source and campaign); confirmed with the instructor
A team member becomes unavailable	Medium	Medium	Shared documentation, code reviews through pull requests, each task paired with a backup member
Secrets pushed to GitHub	High	Low	.gitignore rules, no connection strings in code, Microsoft Entra sign-in instead of passwords

3.3 Stakeholder Analysis

Table (3.7): Stakeholders and Their Needs

Stakeholder	Role	Needs from the platform
Store management	Decides on products, marketing, and budget	Monthly revenue, profit, and refund figures they can trust
Marketing team	Runs paid campaigns (search, social)	Sessions, orders, and conversion rate by traffic source, campaign, and device
Data analysts / BI developers	Build reports and dashboards	Clean, documented star schema queryable with SQL and Power BI
Data engineering team (us)	Builds and runs the platform	Repeatable pipeline, clear logs, quality checks, low cost
Technical instructor (DEPI)	Evaluates the project	Working pipeline, documentation, evidence of every requirement

3.4 Requirements Gathering

3.4.1 User Requirements

The user requirements in Table (3.8) summarize, in the users' own terms, what the stakeholders in Table (3.7) need. They are derived from the stakeholder needs and user stories and are traced to the functional requirements.

Table (3.8): User Requirements

ID	User requirement	Stakeholder	Related FR
UR-01	See monthly revenue, profit, and refunds by product	Store management	FR-08, FR-13
UR-02	See sessions, orders, and conversion rate by traffic source, campaign, and device	Marketing team	FR-08, FR-13
UR-03	Query one clean, documented star schema with SQL or Power BI without cleaning data first	Data analysts / BI developers	FR-08, FR-09, FR-10
UR-04	Explain every difference in row counts through logged rejected records with reasons	Data engineering team	FR-05, FR-06
UR-05	Run the whole pipeline repeatedly and see the status of every step	Data engineering team, instructor	FR-11
UR-06	Verify the solution works end to end with evidence for every requirement	Technical instructor	FR-11, FR-12

3.4.2 Functional Requirements

Table (3.9): Functional Requirements

ID	Requirement
FR-01	Upload the raw CSV files (6 tables) to the raw zone of ADLS Gen2 without changing them.
FR-02	Organize the data lake into raw, processed, and curated zones.
FR-03	Query raw files directly with Synapse Serverless SQL.
FR-04	Clean the data with Python, pandas, and NumPy: remove duplicates, standardize dates, prices, text values, and booleans, and handle nulls.
FR-05	Validate business rules: no negative quantities or prices, refunds not above the item price, no refund dated before its order.
FR-06	Check referential integrity and route orphan records to a rejected-records table with a reason code.
FR-07	Write cleaned and modeled data as Parquet files in processed and curated, using a Parquet-compatible engine such as PyArrow (to be verified during implementation).
FR-08	Build the star schema: FactSales at order-line grain, FactRefunds, and the dimensions DimDate, DimProduct, DimCustomer, and DimStore (online channel).
FR-09	Create external tables and views in Synapse over the curated data.
FR-10	Load the final model into the warehouse.
FR-11	Orchestrate all steps in one pipeline and monitor each run.
FR-12	Apply role-based access control so each member has only the access their role needs.
FR-13	Provide sample analytical queries: sales by product, channel, and month; conversion rate by source.

3.4.3 Non-Functional Requirements

Table (3.10): Non-Functional Requirements

Category	Requirement
Data quality	100% of injected issues detected; row counts reconcile at every stage (rows in = rows loaded + rows rejected).
Performance	A full pipeline run over the 1.74 million records completes in under 30 minutes.
Cost	Total spend stays within the $100 Azure for Students credit; budget alerts at 50%, 80%, and 100%; compute auto-pauses when idle.
Security	Sign-in with Microsoft Entra ID only; Contributor access scoped to one resource group; no secrets or connection strings in GitHub.
Reliability	The pipeline can be re-run from the start and gives the same result (idempotent loads; fixed random seed for test data).
Maintainability	Code in GitHub with one branch per member and pull-request reviews; README in every folder.
Usability	Analysts query clear, documented table and column names; a data dictionary is provided. A graphical user interface is not applicable to this project.
Traceability	Every record in the warehouse can be traced back to its source file and load run.

3.4.4 System and Domain Requirements

System requirements describe the platform on which the solution runs, and domain requirements describe the retail rules the data must obey. Both are taken from the project plan and requirements above and are listed in Table (3.11). Items marked "to be confirmed" depend on the Azure proof of concept.

Table (3.11): System and Domain Requirements

ID	Type	Requirement
SYS-01	System	Microsoft Azure subscription (Free Trial for learning and proof of concept; Azure for Students for the final build), with all resources in one resource group.
SYS-02	System	ADLS Gen2 storage for the raw, processed, and curated zones.
SYS-03	System	Python environment with pandas, NumPy, and a Parquet-compatible library such as PyArrow; exact versions and the host that runs the scripts (local machine or Azure-hosted compute) are to be confirmed.
SYS-04	System	Synapse workspace with a Serverless SQL endpoint for external tables and views; Synapse or Data Factory pipelines for orchestration.
SYS-05	System	A warehouse that stores data and supports table loads (Azure SQL Database is the listed resource and the fallback; final choice to be confirmed by the proof of concept).
SYS-06	System	GitHub repository with one branch per member, pull requests, and no secrets committed.
DOM-01	Domain	The sales fact table has one row per product per order (order line).
DOM-02	Domain	Quantities and prices are never negative; a refund never exceeds the item price; a refund is never dated before its order.
DOM-03	Domain	Every fact record refers to existing dimension members; orphan records are rejected with a reason code.
DOM-04	Domain	The data covers March 2012 to March 2015 and a small product catalog (4 products).
DOM-05	Domain	The dataset has no physical stores, so the store dimension represents the online sales channel (traffic source and campaign).

3.5 Use Cases and User Stories

3.5.1 User Stories

1. As a **store manager**, I want monthly revenue, profit, and refunds by product, so that I can see which products to promote.
2. As a **marketing analyst**, I want conversion rate by traffic source and device, so that I can move budget to the best campaigns.
3. As a **BI developer**, I want one documented star schema, so that I can build dashboards without cleaning data myself.

4. As a **data engineer**, I want every rejected record logged with its reason, so that I can explain every difference in row counts.
5. As the **instructor**, I want screenshots of a full monitored pipeline run, so that I can verify the solution works end to end.

3.5.2 Use Cases

Table (3.12): Use Cases

ID	Use case	Actor	Main flow	Result
UC-01	Ingest raw data	Data engineer	Uploads the 6 CSV files to raw; the pipeline is started	Raw files stored unchanged and queryable with Serverless SQL
UC-02	Clean and validate data	Pipeline (run by the data engineer)	Reads raw, removes duplicates, standardizes formats, checks business rules and keys, writes Parquet to processed	Clean data plus a rejected-records table with reason codes
UC-03	Review rejected records	Data quality engineer	Compares rejected and fixed records with injection_manifest.csv and reconciles row counts	Test result: issues caught versus issues injected
UC-04	Build and load the star schema	Pipeline (run by the SQL engineer)	Builds facts and dimensions in curated, creates external tables and views, loads the warehouse	Star schema available in the warehouse
UC-05	Analyze sales and marketing	Store manager, marketing analyst, BI developer	Queries the warehouse with SQL or Power BI: sales by product, channel, and month; conversion by source	Reports and answers to business questions
UC-06	Monitor a pipeline run	Data engineer, instructor	Opens the monitoring view and checks each step's status and duration	Evidence of a successful end-to-end run
UC-07	Manage access	Azure administrator	Adds members to the security group and assigns roles on the resource group	Each member has only the access their role needs

DIAGRAM PLACEHOLDER

To be replaced by the UML use-case diagram showing the actors (data engineer, data quality engineer, store manager, marketing analyst, BI developer, instructor, Azure administrator) and use cases UC-01 to UC-07.

Conceptual placeholder only — not an implemented system diagram.

Fig (3.1): Use-Case Diagram (UC-01 to UC-07)

3.6 Key Performance Indicators

The KPIs in Table (3.13) define how the success of the project is measured. They are the acceptance criteria used by the test plan in Chapter Five.

Table (3.13): Key Performance Indicators

KPI	Target	How it is measured
Injected issues detected	100%	Pipeline rejects and fixes compared with injection_manifest.csv
Row-count reconciliation	0 unexplained differences	Rows in = rows loaded + rows rejected, for every table and stage
End-to-end pipeline run	1 successful run with no manual steps	Synapse or Data Factory monitoring screenshots
Referential integrity in the warehouse	0 orphan foreign keys	SQL checks on every fact-to-dimension key
Cloud cost	Within the $100 student credit	Azure Cost Management report
Milestones delivered on time	4 of 4	Submission dates against the plan
Individual contribution	Every member has merged commits	GitHub contributors page

CHAPTER FOUR

CHAPTER 4 System Design

*This chapter describes the **planned** design of the platform. Unless a statement says otherwise, the designs below are proposals to be confirmed in Phase 2 (dataset profiling and Azure proof of concept) and are not yet implementation results.*

4.1 System Architecture Overview

The platform moves data through the following stages. Source CSV files are uploaded unchanged into the **raw zone** of ADLS Gen2. A Python-based ETL pipeline built with Python, pandas, and NumPy reads the raw files, cleans and validates them, and writes Parquet files to the **processed zone**, together with logs of rejected records. A second transformation step builds the fact and dimension tables of the star schema and writes them as Parquet to the **curated zone**. Synapse Serverless SQL exposes the lake through external tables and views, and the final model is loaded into a **warehouse** that stores data. Analysts query the warehouse with SQL or Power BI. A pipeline orchestrates and monitors all steps, and Microsoft Entra ID with role-based access control secures them.

Table (4.1): Architecture Layers and Components

Layer	Components	Role
Source	Maven Fuzzy Factory CSV files (6 tables); dirty-data generator script and injection manifest	Provides the data and the known answer key
Storage (data lake)	ADLS Gen2: raw, processed, curated zones	Holds data at each stage of quality
Processing	Python-based ETL pipeline: Python, pandas, NumPy, with a Parquet-compatible engine such as PyArrow	Cleans, validates, transforms, and writes Parquet
Query layer	Synapse Serverless SQL: external tables and views	Queries lake files with T-SQL without loading them
Warehouse	Azure SQL Database (listed resource; final choice confirmed by the proof of concept)	Stores the final star schema and supports table loads
Orchestration and monitoring	Synapse or Data Factory pipelines	Runs all steps in order and records status and duration
Security and governance	Microsoft Entra ID, RBAC, budget alerts, GitHub	Controls access, cost, and code changes
Consumption	SQL queries and Power BI	Reports and answers to business questions

DIAGRAM PLACEHOLDER

To be replaced by the architecture diagram: source CSV → ADLS Gen2 (raw → processed → curated) with the Python-based ETL pipeline between zones → Synapse Serverless SQL → warehouse → SQL / Power BI, with orchestration, monitoring, and Entra ID / RBAC.

Conceptual placeholder only — not an implemented system diagram.

Fig (4.1): System Architecture Diagram

4.2 Design Constraints and Assumptions

Constraints

- **Budget:** total spend must stay within the $100 Azure for Students credit; the $200 Free Trial credit (until about 9 Nov 2026) is used only for learning and the proof of concept.
- **Services:** no Dedicated SQL Pool and no always-on services; compute must auto-pause when idle; all resources are kept in one resource group.
- **Processing style:** batch only; there is no real-time or streaming ingestion.
- **Processing technology:** data cleaning and transformation use Python, pandas, and NumPy; Parquet output requires a compatible engine such as PyArrow.
- **Serverless SQL:** Synapse Serverless SQL is a query layer over external data and cannot store tables or run INSERT, UPDATE, or DELETE statements, so a separate warehouse is required.
- **Security:** Microsoft Entra ID sign-in only; no secrets or connection strings in GitHub.
- **Time:** fixed milestone dates between 19 Oct and 4 Dec 2026 (Table 3.1).

Assumptions and dependencies

- The dataset can be profiled and approved early (it is on the critical path), and the injected issues are fully documented in the injection manifest.
- The student subscription quotas allow the required Azure services; the proof of concept checks this early, with Azure SQL Database as the fallback warehouse.
- Each table fits in memory on the machine that runs the Python scripts when processed one at a time. This is to be verified during profiling.

- The instructor confirms that modeling the store dimension as the online sales channel is acceptable.

4.3 Data Source and Dataset Description

The source is the Maven Fuzzy Factory toy-store e-commerce dataset [7], which contains six tables and 1,735,068 records for the period March 2012 to March 2015. Table (4.2) describes the tables. The record counts and key columns come from the public dataset description and are to be confirmed during dataset profiling in Phase 2.

Table (4.2): Source Dataset Tables

Table	Description	Approx. records	Key columns
website_pageviews	Page views within website sessions	1,188,124	website_pageview_id, website_session_id, pageview_url, created_at
website_sessions	Website visits with traffic source, campaign, and device	472,871	website_session_id, user_id, utm_source, utm_campaign, device_type, created_at
order_items	Products on each order (order lines)	40,025	order_item_id, order_id, product_id, price_usd, cogs_usd
orders	Customer orders	32,313	order_id, website_session_id, user_id, items_purchased, price_usd, cogs_usd
order_item_refunds	Refunds of order items	1,731	order_item_refund_id, order_item_id, order_id, refund_amount_usd
products	Product catalog (4 products)	4	product_id, product_name, created_at

Injected data-quality issues. The original data is clean. A reproducible, seeded script injects 32 documented issue types and writes an injection manifest (`injection_manifest.csv`) that records what was injected. The manifest is the answer key for testing: the pipeline must detect and fix or reject 100% of the injected issues. The issue types fall into the categories handled by the cleaning rules (Section 4.7): duplicates, inconsistent formats, null values, business-rule violations, and orphan records.

4.4 Data Lake Architecture

The data lake is organized into three zones in ADLS Gen2, following the medallion pattern (Table 2.1). Each zone has a distinct purpose, format, and write rule, summarized in Table (4.3).

Table (4.3): Data Lake Zones

Zone	Content	Format	Rule
Raw	The six source CSV files exactly as received	CSV	Never modified after upload
Processed	Cleaned, validated tables and rejected-record logs	Parquet	Written only by the Python-based ETL pipeline
Curated	Fact and dimension tables of the star schema	Parquet	Written only by the modeling step; source for external tables and the warehouse load

4.4.1 Raw Zone

The raw zone stores the six CSV files without change, so the original data can always be reproduced and compared with the cleaned output. Raw files can be queried directly with Synapse Serverless SQL (FR-03).

4.4.2 Processed Zone

The processed zone stores the cleaned and validated version of each table as Parquet files, with standardized data types and formats. The rejected-record logs are stored here as well, each record carrying a reason code.

4.4.3 Curated Zone

The curated zone stores the modeled data, namely the fact tables and dimension tables, as Parquet files. External tables and views in Synapse Serverless SQL point to this zone, and the warehouse is loaded from it.

DIAGRAM PLACEHOLDER

To be replaced by a diagram of the raw, processed, and curated zones with the data flow and the transformation step between each pair of zones, and the rejected-record logs.

Conceptual placeholder only — not an implemented system diagram.

Fig (4.2): Data Lake Zones and Data Flow

4.5 Data Ingestion Design

Ingestion is a batch process (UC-01, FR-01). The six CSV files are uploaded to the raw zone without any change and the pipeline is then started. The upload mechanism (for example, the

Azure portal, a command-line tool, or a pipeline copy activity) is to be selected during the proof of concept. After upload, the file names, row counts, and load run identifier are recorded so that every record can be traced to its source file and load run (traceability requirement).

4.6 Python-Based Data Cleaning and Transformation Design

Cleaning and transformation are performed by a Python-based ETL pipeline made of Python scripts that use pandas DataFrames and NumPy operations. Each source table is processed as a separate batch step, which keeps memory use bounded. The stages are shown in Table (4.4).

Table (4.4): Python-Based Cleaning and Transformation Stages

Stage	Description	Main tools
1. Read	Read a raw CSV file into a pandas DataFrame with explicit column types	pandas
2. De-duplicate	Remove duplicate rows and duplicate keys	pandas
3. Standardize	Convert dates, prices, text values, and booleans to one standard format	pandas, NumPy
4. Handle nulls	Fill, derive, or reject records with missing values according to the rules	pandas, NumPy
5. Validate rules	Check business rules (no negative quantities or prices, refunds not above the item price, no refund before its order) using vectorized conditions	pandas, NumPy
6. Check references	Check that order items, orders, refunds, and sessions refer to existing parent records; route orphans to the rejected table with a reason code	pandas
7. Write	Write clean data and rejected records as Parquet to the processed zone	pandas with a Parquet-compatible engine (such as PyArrow)
8. Reconcile	Check that rows in = rows loaded + rows rejected and store the counts	pandas
9. Model	Build the fact and dimension tables from the processed data and write them to the curated zone	pandas, NumPy

Design principles of the Python-based ETL pipeline:

- **Rules are code:** each cleaning and validation rule is a named, testable function with a reason code.
- **Idempotent runs:** re-running from the start gives the same result; outputs are overwritten, not appended.
- **No secrets in code:** access to storage uses Microsoft Entra ID sign-in, not connection strings.

- **Clear separation:** pandas handles tabular operations and NumPy handles vectorized numeric and conditional checks. Neither writes Parquet by itself; a compatible engine such as PyArrow is required, and its use is subject to verification during implementation.
- **Execution host:** the machine or Azure service that runs the scripts, and how the pipeline calls them, is to be confirmed in the proof of concept.

4.7 Data Quality Validation Framework

Data quality is measured, not assumed. The framework has four parts: rules, rejected-record logging, row-count reconciliation, and comparison with the injection manifest.

Table (4.5): Data Quality Rule Categories

Category	Examples of rules	Handling	Req.
Duplicates	Duplicate rows or duplicate primary keys	Remove; count	FR-04
Format standardization	Dates, prices, text values, and booleans in inconsistent formats	Convert to standard format	FR-04
Nulls	Missing values in required columns	Fill, derive, or reject by rule	FR-04
Business rules	No negative quantities or prices; refund not above the item price; no refund dated before its order	Reject with reason code	FR-05
Referential integrity	Order items without an order; refunds without an order item	Reject to rejected-records table with reason code	FR-06

- **Rejected-record logging:** every rejected record is stored with its source table, source file, record key, rule code, reason, and load run identifier.
- **Row-count reconciliation:** for every table and stage, rows in must equal rows loaded plus rows rejected; the target is 0 unexplained differences.
- **Answer-key comparison:** the pipeline's fixes and rejects are compared with `injection_manifest.csv`; the target is 100% of injected issues detected.

4.8 Dimensional Modeling

The model follows Kimball dimensional modeling [2]. The grain is declared first: **one row per product per order (order line)**. The tables below are the proposed design; the exact attributes and keys are finalized in Phase 2 after dataset profiling.

4.8.1 Fact Tables

Table (4.6): Fact Tables (Proposed)

Fact table	Grain	Keys	Measures
FactSales	One row per order line (product per order)	DateKey, ProductKey, CustomerKey, StoreKey; degenerate order_id and order_item_id	Price (USD), cost of goods sold (USD), gross margin (price minus cost), primary-item flag
FactRefunds	One row per refunded order item	DateKey, ProductKey, CustomerKey, StoreKey; order_id and order_item_id	Refund amount (USD)

4.8.2 Dimension Tables

Table (4.7): Dimension Tables (Proposed)

Dimension	Description	Example attributes
DimDate	Calendar dates covering March 2012 to March 2015	Date, day, week, month, quarter, year
DimProduct	The product catalog (4 products)	Product id, product name, launch date
DimCustomer	Customers identified in orders and sessions	Customer (user) id, repeat-customer indicator
DimStore	The online sales channel, because the dataset has no physical stores	Traffic source, campaign, device type

4.8.3 Star Schema Design

FactSales and FactRefunds sit at the center and join to DimDate, DimProduct, DimCustomer, and DimStore through surrogate keys. All fact-to-dimension keys must resolve to an existing dimension row, which is tested as 0 orphan foreign keys (Table 3.13).

DIAGRAM PLACEHOLDER

To be replaced by the star-schema diagram showing FactSales and FactRefunds at the center joined to DimDate, DimProduct, DimCustomer, and DimStore, with primary and foreign keys.

Conceptual placeholder only — not an implemented system diagram.

Fig (4.3): Star Schema Diagram

Open design question (team input required). The sample query "conversion rate by source" (FR-13) needs session counts as well as orders. FactSales and FactRefunds do not hold sessions, so the team must decide whether to add a session-level fact or a pre-aggregated sessions table

to the curated zone. This decision is to be taken during Phase 2 and recorded in the data dictionary.

4.9 Warehouse Architecture

The data lake and the warehouse are different things and must not be confused. The curated zone is storage of files; Synapse Serverless SQL is a query layer over those files; and the warehouse is a database that stores the final model. Table (4.8) compares them.

Table (4.8): Roles of the Curated Zone, Serverless SQL, and the Warehouse

Component	Stores data?	Supports INSERT / UPDATE / DELETE?	Role
Curated zone (ADLS Gen2)	Yes (Parquet files)	Files are written by the pipeline	Holds the modeled fact and dimension tables
Synapse Serverless SQL	No (queries external data)	No	External tables and views over the lake for exploration and querying [3]
Warehouse (Azure SQL Database)	Yes (database tables)	Yes	Holds the final star schema that analysts and Power BI query

The warehouse is loaded from the curated zone as part of the pipeline (FR-10) using idempotent loads, so a re-run produces the same result. The loading method (for example, a pipeline copy activity or SQL-based load) and the final warehouse product are to be confirmed by the proof of concept; Azure SQL Database is the listed resource and the fallback if student quotas block other options.

4.10 Pipeline Orchestration

A single pipeline in Synapse or Data Factory orchestrates all steps (FR-11): upload or register raw files, run the Python-based cleaning and validation, write processed data, build and write curated data, refresh external tables and views, load the warehouse, and run checks. Each step reports its status and duration in the monitoring view, which provides the evidence for the end-to-end run KPI. The pipeline can be re-run from the start (reliability requirement), and compute auto-pauses when idle to control cost.

DIAGRAM PLACEHOLDER

To be replaced by the pipeline flow diagram showing the ordered steps, their dependencies, the quality checks, and the monitoring points.

Conceptual placeholder only — not an implemented system diagram.

Fig (4.4): Pipeline Orchestration Flow

4.11 Security and Role-Based Access Control

Security follows the principle of least privilege (FR-12) and is summarized in Table (4.9).

Table (4.9): Security Controls

Area	Control
Authentication	Sign-in with Microsoft Entra ID only; no passwords or connection strings in code
Authorization	Team members are added to a security group; roles are assigned on the resource group so each member has only the access their role needs; Contributor access is scoped to one resource group
Secrets	.gitignore rules; no secrets or connection strings in GitHub
Cost governance	Budget alerts at 50%, 80%, and 100%; one resource group that can be deleted at once; no Dedicated SQL Pool
Code governance	One branch per member and pull-request reviews

4.12 System Diagrams

The plan includes architecture, entity-relationship, data-flow, and UML diagrams (Table 3.2). The architecture, zone, star-schema, and pipeline diagrams appear in the sections above. The remaining diagrams are listed here as placeholders and are replaced with the actual diagrams when they are produced in Phase 2.

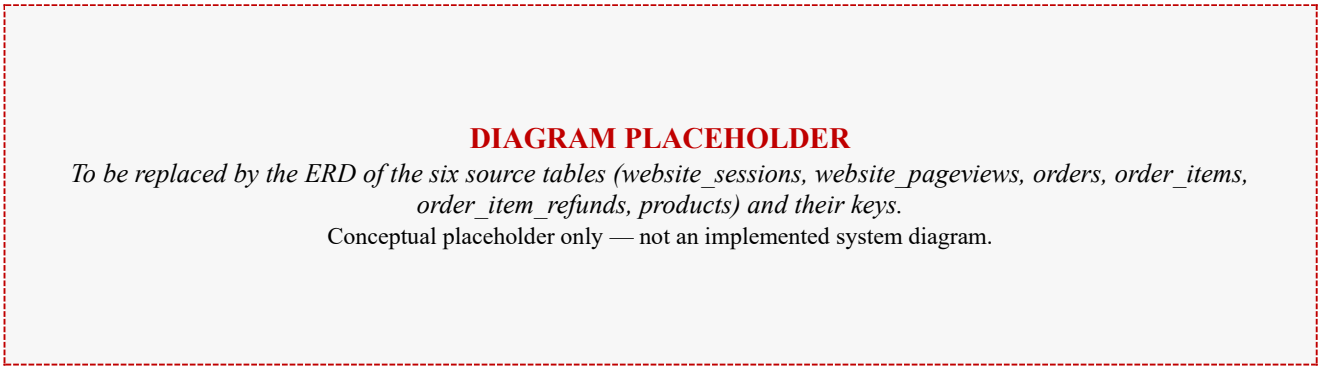


Fig (4.5): Entity Relationship Diagram of the Source Dataset

DIAGRAM PLACEHOLDER

To be replaced by the data-flow diagram of the platform: external entities, processes (ingest, clean and validate, model, load, query), and data stores (raw, processed, curated, warehouse).

Conceptual placeholder only — not an implemented system diagram.

Fig (4.6): Data Flow Diagram (Level 0 and Level 1)

DIAGRAM PLACEHOLDER

To be replaced by a UML sequence diagram of a pipeline run: data engineer, pipeline, ADLS Gen2, Python ETL scripts, Serverless SQL, and warehouse.

Conceptual placeholder only — not an implemented system diagram.

Fig (4.7): UML Sequence Diagram of an End-to-End Pipeline Run

CHAPTER FIVE

CHAPTER 5 Implementation and Testing

Status of this chapter. *This chapter describes the development environment, the planned implementation of each component, and the test plan. At the time of writing, the implementation phase (7–30 Nov 2026) has not been completed, so no code, screenshots, deployment results, or test outcomes are reported. Each result is to be added only after it has been produced and verified. Planned work is labeled as such.*

Table (5.1): Implementation Status Register

Component	Planned approach	Status	Evidence to add
Development environment	GitHub repository; Python with pandas, NumPy, and a Parquet-compatible library	Repository created; environment to be confirmed	Dependency list, repository link
Python ETL implementation	Python scripts per table, one rule per function	Planned	Source code in repository
Data cleaning with pandas	De-duplication, standardization, null handling	Planned	Rule list, before/after counts
Numerical processing with NumPy	Vectorized business-rule and numeric checks	Planned	Rule list, test results
Validation and rejected records	Rejected-record table with reason codes	Planned	Rejected-record sample, counts
Parquet generation	Write processed and curated Parquet via a compatible engine	Planned; engine to be verified	File listing, schema
Azure Data Lake integration	Raw, processed, curated zones in ADLS Gen2	Planned (proof of concept in Phase 2)	Portal screenshots
SQL transformation and warehouse load	External tables, views, warehouse load	Planned	SQL scripts, load logs
Pipeline monitoring	Pipeline with per-step status	Planned	Monitoring screenshots

5.1 Development Environment

The code, SQL scripts, and documentation are kept in the team's GitHub repository, with one branch per member and pull-request reviews. The Python environment uses Python, pandas, NumPy, and a Parquet-compatible library such as PyArrow. Library versions and the host that runs the scripts are to be recorded here once confirmed.

5.2 Python ETL Implementation

Planned. The ETL is implemented as Python scripts that follow the stages of Table (4.4), with one function per rule and a common run identifier. Code excerpts and the repository structure are to be added after implementation.

5.3 Data Cleaning with pandas

Planned. pandas DataFrames are used to read the raw files, remove duplicates, standardize formats, handle nulls, and check references. Before-and-after row counts for each table are to be reported here.

5.4 Numerical Processing with NumPy

Planned. NumPy is used for vectorized numeric and conditional checks, such as detecting negative quantities and prices or refunds above the item price, and for computing derived measures. Results are to be added after implementation.

5.5 Data Validation and Rejected Records

Planned. The rejected-records table and its reason codes are to be documented, together with the reconciliation of rows in, rows loaded, and rows rejected for every table and stage, and the comparison with `injection_manifest.csv`.

5.6 Parquet File Generation

Planned. Parquet files are written to the processed and curated zones through a compatible engine. The engine, compression, and resulting schemas are to be verified and reported.

5.7 Azure Data Lake Integration

Planned. The ADLS Gen2 account, containers, and zone folders are to be created during the proof of concept and documented with screenshots, together with the access-control assignments.

5.8 SQL Transformation and Warehouse Loading

Planned. The SQL scripts for external tables, views, and the warehouse load are to be documented, including the checks for orphan foreign keys and the sample analytical queries (FR-13).

5.9 Pipeline Monitoring

Planned. Monitoring screenshots of a complete pipeline run, showing the status and duration of each step, are to be added as evidence for the end-to-end KPI.

5.10 Testing and Validation

The test plan in Table (5.2) derives its acceptance criteria from the KPIs (Table 3.13) and non-functional requirements (Table 3.10). All tests are **not yet executed**; results are to be recorded here after Phase 3.

Table (5.2): Test Plan

ID	Objective	Method	Pass criterion	Status
T-01	All injected issues are detected	Compare rejects and fixes with injection_manifest.csv	100% detected	Not yet executed
T-02	Row counts reconcile	Rows in = rows loaded + rows rejected, per table and stage	0 unexplained differences	Not yet executed
T-03	Parquet output is valid	Read processed and curated files back and compare schema and counts; query through Serverless SQL	Files readable; schema as designed	Not yet executed
T-04	Referential integrity in the warehouse	SQL checks on every fact-to-dimension key	0 orphan foreign keys	Not yet executed
T-05	End-to-end run	Run the pipeline without manual steps; capture monitoring	1 successful run	Not yet executed
T-06	Idempotent re-run	Run the pipeline twice from the start and compare outputs	Identical results	Not yet executed
T-07	Run time	Time a full run over 1.74M records	Under 30 minutes	Not yet executed
T-08	Access control	Check each role's access on the resource group	Only access needed by the role	Not yet executed
T-09	Sample analytical queries	Run FR-13 queries and check results against the cleaned data	Correct results	Not yet executed

5.11 Performance and Cost Evaluation

Performance (full pipeline run time and memory use of the Python scripts) and cost (spend against the $100 student credit) are to be measured and reported here after implementation, using the pipeline monitoring view and the Azure Cost Management report. No performance or cost figures are claimed at this stage.

Table (5.3): Performance and Cost Results (to be completed)

Measure	Target	Measured result
Full pipeline run time (1.74M records)	Under 30 minutes	To be measured
Peak memory of the Python ETL	No fixed target; recorded for reference	To be measured
Total cloud spend	Within the $100 student credit	To be measured

REFERENCES

[1] Microsoft Learn. Implement medallion lakehouse architecture in Fabric.

[2] Kimball Group. Dimensional modeling techniques: Grain.

[3] Microsoft Learn. Serverless SQL pool in Azure Synapse Analytics.

[4] Azure Architecture Center. Analytics end-to-end with Microsoft Fabric.

[5] Microsoft (GitHub). Azure Analytics End to End with Azure Synapse — Deployment Accelerator.

[6] D. Palli, DEV Community (2025). Completed: End-to-End Data Engineering Project on Microsoft Azure.

[7] Maven Analytics. Data Playground — Toy Store E-Commerce Database.